

OBSERVATION TASK 2

Field	Details
Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	Task 2
Student Name	R . Varshini
Roll No.	A24126552115
Date	30-08-2026

1. TITLE

Dynamic Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To create a JavaScript application that uses a class to represent a student, accepts user input through a form, and dynamically generates and displays the student's profile on the webpage using DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To define a JavaScript class with properties such as Name, Roll Number, Department and CGPA.
- To create an object of the class using user-provided values.
- To select DOM elements using `getElementById()`.
- To handle a button click event using `addEventListener()`.
- To dynamically create HTML elements using `createElement()` and set their content.
- To add the dynamically created elements to the webpage using `appendChild()`.
- To apply basic CSS styling to the dynamically generated profile.

4. THEORY

A JavaScript class is a template for creating objects that bundle related data (properties) and behavior together. The `constructor()` method runs automatically when a new object is created with the `new` keyword, and it is used to initialize the object's properties from the arguments passed to it.

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Methods such as `document.getElementById()` are used to select a specific element from this tree, while properties like `.value` read the current content of an input field.

Elements can also be created entirely from JavaScript using `document.createElement()`, which returns a new, unattached element. Its text can be set using the `textContent` property, and it is inserted into the visible page using `parentElement.appendChild(childElement)`.

Event handling connects user actions to code. `button.addEventListener("click", callback)` registers a function that runs every time the button is clicked, allowing the page to react to user input without reloading.

In this experiment, these concepts are combined: a `Student` class stores the entered details as an object, and a click event handler reads the input values, creates the object, and builds the profile display dynamically using DOM methods.

5. ALGORITHM / PROCEDURE

- Create an HTML file with input fields for Name, Roll Number, Department and CGPA, a Display button, and an empty container for the profile.
- Link an external CSS file to style the form and profile container.
- Link an external JavaScript file to the HTML page.
- Define a Student class with a constructor that accepts name, rollNumber, department and cgpa.
- Select the Display button using document.getElementById().
- Attach a click event listener to the button using addEventListener().
- Inside the event handler, read the values of all input fields using .value.
- Create a new Student object using the values read from the form.
- Select the profile container element and clear any previous content.
- Dynamically create heading and paragraph elements using createElement().
- Set the text content of each created element using the student object's properties.
- Append all created elements to the profile container using appendChild().
- Open the HTML file in a browser, enter sample student details, and click the button.
- Verify that the profile is displayed correctly and repeat with different input values.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Profile</title>

  <!-- Link CSS file -->
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1>Student Profile</h1>

    <input type="text" id="name" placeholder="Enter Name">
    <input type="text" id="roll" placeholder="Enter Roll Number">
    <input type="text" id="department" placeholder="Enter Department">
    <input type="number" id="cgpa" step="0.01" placeholder="Enter CGPA">

    <br>

    <button id="displayBtn">Display Student Profile</button>

    <div id="profile"></div>

  </div>

  <!-- Link JavaScript file -->
  <script src="script.js"></script>

</body>
</html>
```

style.css

```

body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  text-align: center;
  padding: 30px;
}

.container {
  background-color: white;
  width: 400px;
  margin: auto;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0 0 10px gray;
}

input {
  width: 90%;
  padding: 10px;
  margin: 8px;
}

button {
  padding: 10px 20px;
  background-color: blue;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

#profile {
  margin-top: 20px;
  padding: 20px;
  background-color: #e8f0fe;
  border-radius: 10px;
  text-align: left;
}

#profile h2 {
  text-align: center;
}

```

script.js

```

class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = cgpa;
  }
}

// Select button using DOM
const button = document.getElementById("displayBtn");

// Event handling
button.addEventListener("click", function () {

  // Get values from input fields
  let name = document.getElementById("name").value;
  let roll = document.getElementById("roll").value;
  let department = document.getElementById("department").value;
  let cgpa = document.getElementById("cgpa").value;

  // Create Student object
  let student = new Student(

```

```

        name,
        roll,
        department,
        cgpa
    );

    // Select profile area
    let profile = document.getElementById("profile");

    // Clear previous content
    profile.innerHTML = "";

    // Create elements dynamically
    let heading = document.createElement("h2");
    heading.textContent = "Student Profile";

    let nameElement = document.createElement("p");
    nameElement.textContent = "Name      : " + student.name;

    let rollElement = document.createElement("p");
    rollElement.textContent = "Roll No   : " + student.rollNumber;

    let departmentElement = document.createElement("p");
    departmentElement.textContent =
        "Department : " + student.department;

    let cgpaElement = document.createElement("p");
    cgpaElement.textContent = "CGPA      : " + student.cgpa;

    // Add elements to webpage
    profile.appendChild(heading);
    profile.appendChild(nameElement);
    profile.appendChild(rollElement);
    profile.appendChild(departmentElement);
    profile.appendChild(cgpaElement);
});

```

7. CODE EXPLANATION

Code	Explanation
class Student	Defines a blueprint for a student object with name, rollNumber, department and cgpa properties.
constructor()	Initializes a new Student object's properties when it is created with new.
document.getElementById()	Selects a single DOM element by its id attribute, e.g. the button or an input field.
.value	Reads the current text entered by the user in an input field.
addEventListener("click", fn)	Registers a function to run whenever the button is clicked (event handling).
new Student(...)	Creates an object of the Student class using the values read from the form.
document.createElement()	Creates a new HTML element (e.g. <h2> or <p>) that is not yet part of the page.
.textContent	Sets or reads the visible text inside an element.
appendChild()	Inserts a created element into the page as a child of another element (the profile container).
innerHTML = ""	Clears any previously displayed profile before showing the new one.

8. TEST CASES AND EXECUTION DATA

Each test case was executed by entering the given values into the form and clicking the Display Student Profile button.

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Status
TC-01	Ravi Kumar	A23126552001	CSE (AI & ML)	8.42	Profile displayed with entered details	Pass

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Status
TC-02	Priya Sharma	A23126552002	CSE (Data Science)	9.10	Profile updates to new details	Pass
TC-03	Arjun Singh	A23126552003	ECE	7.55	Profile displayed correctly	Pass
TC-04	Kiran Reddy	A23126552004	CSE (Cyber Security)	0.00	Profile displayed with 0.00 CGPA	Pass

Additional Validation Test

Test Case	Input	Expected Result	Status
TC-05	All fields left empty, button clicked	Profile is generated with blank values (no fields are marked required)	Pass

9. OUTPUT

Output 1: Student Profile Form

The webpage displays a styled container with input fields for Name, Roll Number, Department and CGPA, and a "Display Student Profile" button.

Output 2: Generated Profile

After entering the details and clicking the button, a profile section is dynamically created below the form showing the entered student details.

The image shows a web application interface for displaying a student profile. It consists of two main parts: a form for input and a display area for the resulting profile.

Form Section:

- Title:** Student Profile
- Fields:**
 - Name: RUTHALA VARSHINI
 - Roll No: A24126552115
 - Department: CSM
 - CGPA: 8.6
- Action:** A blue button labeled "Display Student Profile".

Display Section:

- Title:** Student Profile
- Fields:**
 - Name : RUTHALA VARSHINI
 - Roll No : A24126552115
 - Department : CSM
 - CGPA : 8.6

10. RESULT

Thus, a JavaScript application was successfully developed that defines a Student class, creates an object from user-entered form data, and dynamically generates and displays the student's profile on the webpage using DOM manipulation and event handling. The application was verified using multiple test cases.